Samsung Research

01.

02.

03.

02. K Element Sort

Problem statement

Consider an array of size n is grouped such that every group has k elements and number formed by each group using the digits from left to right is:
arr[j]arr[j+1]arr[j+2]...arr[j+k-1] (where j is the start index of a group of the array)
For example :
array : 1, 32, 5, 4, 1, 2
k : 2
As value of k is 2, number formed by these groups will be 132, 54 and 12.
For finding k element sort you have to sort the above numbers in ascending order and return the original array. In the above case,
After sorting these numbers in ascending order, we will get 12, 54, 132 and final array become 1, 2, 5, 4, 1, 32.

You are given a function,

```
int* KElementSort(int* arr, int n, int k);
```

The function accepts an integer array 'arr' of length 'n' and an integer 'k' as its arguments. Implement the function to return the array after applying k element sorting technique.

Assumptions:

1. k > 0 such that number formed by combining k elements never cross the limit of integer.
2. n > 0 and is multiple of k.
3. Array index starts from 0.
4. All elements of array are >= 0

Sample Input

```
k: 3
n: 9
arr : 1 7 2 9 1 3 1 5 3
```

Sample Output

```
1 5 3 1 7 2 9 1 3
Explanation:
After applying k element sorting technique this will be final array.

The custom input format for the above case:
```

01.
02.
03.

```
k: 3
n: 9
arr : 1 7 2 9 1 3 1 5 3
```

Sample Output

```
1 5 3 1 7 2 9 1 3
Explanation:
After applying k element sorting technique this will be final array.

The custom input format for the above case:
3
9
1 7 2 9 1 3 1 5 3
(The first line represents 'k' , the second line the length of the 'arr', the third line represents the 'arr')
```

Instructions :

- This is a template based question, DO NOT write the "main" function.
- Your code is judged by an automated system, do not write any additional welcome/greeting messages.
- "Save and Test" only checks for basic test cases, more rigorous cases will be used to judge your code while scoring.
- Additional score will be given for writing optimized code both in terms of memory and execution time.

Now let's start coding :

Language: C (Gcc 7.5)

Day Mode

> Read-only code below . . .

```c
int* KElementSort(int* arr, int n, int k);
int main()
{
    //Input read from STDIN
    int* result = KElementSort(arr, n, k);
    //Values at result printed to STDOUT
    return 0;
}
```

01.
02.
03.

01. Two Binary String sum

Problem statement

The binary number system only uses two digits, 0 and 1. Any string that represents a number in the binary number system can be called a binary string.
You are required to implement the following function:

```
char* TwoBinaryStringSum(char num1[], char num2[]);
```

The function accepts two binary strings, 'num1' and 'num2' as its arguments. Implement the function to return a binary string which is the sum of 'num1' and 'num2'.
Note: You need to process the strings from right to left when computing sum.
Assumption: binary strings 'num1' and 'num2' can not be NULL (None, in case of Python)

Example

Input:

num1: 10001
num2: 1101

Output:

11110

Explanation

The sum of the two binary strings 10001 and 1101 is 11110, hence 11110 is returned.

The custom input format for the above case:

10001
1101

(The first line represents the first binary string 'num1', the second line represents the second binary string 'num2')

Sample Input

```
110
1001
```

Sample Output

```
1111
The custom input format for the above case:
110
1001
(The first line represents the first binary string 'num1', the second line represents the second binary string 'num2')
```

110
1001

01.
02.
03.

Sample Output

```
1111
```

The custom input format for the above case:

```
110
1001
```

(The first line represents the first binary string 'num1', the second line represents the second binary string 'num2')

Instructions :

- This is a template based question, DO NOT write the "main" function.
- Your code is judged by an automated system, do not write any additional welcome/greeting messages.
- "Save and Test" only checks for basic test cases, more rigorous cases will be used to judge your code while scoring.
- Additional score will be given for writing optimized code both in terms of memory and execution time.

Now let's start coding :

Language: C (Gcc 7.5)

Day Mode

Read-only code below . . .

```
char* TwoBinaryStringSum(char num1[], char num2[]);

int main()
{
    //Input read from STDIN
    char *p = TwoBinaryStringSum(num1, num2);
    //Value at p is printed to STDOUT
    return 0;
}

```

Write your code below . . .

```
char* TwoBinaryStringSum(char num1[], char num2[])
{
    /* Write your code here. */
}
```